Bellora

Best online second hand market

Csoporttagok:

- Gere Dániel
- Csák Zoltán Barnabás

Tartalom

Telepítési útmutató: 3
1. A szoftver célja 4
Kinek és miért? 4
2. Fejlesztői dokumentáció 6
Fejlesztőkörnyezet 6
Használt IDE-k: 6
Technikai Információk 6
Frontend 7
Backend 7
Adatbázis 8
3. Adatszerkezetek 8
Adatbázis dokumentációja 8
Táblák és oszlopok 8
1. categories (Kategóriák) 8
2. favorites (Kedvencek) 8
3. itemimages (Termékképek) 9
4. items (Termékek) 9
5. messages (Üzenetek) 9
6. offers (Ajánlatok) 9
7. orders (Rendelések) 10
8. payments (Fizetések) 10
9. reviews (Értékelések) 10
10. shipping (Szállítások) 11
11. reports (Jelentések) 11
12. users (Felhasználók) 11
Kapcsolatok az adatbázisban 12
ER (Entity Relationship) Diagramm 12
4. Tesztek 13
..... 14
Backend felépítése 18
5. Felhasználói dokumentáció 43
6. Összefoglalás 48
7. Irodalom jegyzék 49

Telepítési útmutató:

Előfeltételek:

- Node.js 18+
- MySQL 8.0+
- npm

Backend telepítése

1. Navigálj a backend mappába:

```
cd backend
npm install
```

2. Hozz létre egy `.env` fájlt:

```
PORT=5000
JWT_SECRET=bellora_secret_jwt_key
DATABASE_URL="mysql://root:@localhost:3306/bellora"
```

3. Indítsd el a szervert:

```
npm run dev
```

Frontend telepítése

1. Navigálj a frontend mappába:

```
cd frontend
npm install
```

2. Hozz létre egy `.env` fájlt:

```
VITE_API_URL=http://localhost:5000/api
VITE_BASE_URL=http://localhost:5000
```

3. Indítsd el a frontend alkalmazást:

```
npm run dev
```

1. A szoftver célja

A Bellora egy innovatív online marketplace, amely használt és preloved ruhákat, cipőket, kiegészítőket és divatcikkeket köt össze eladók és vásárlók között. A platform célja, hogy egyszerűbbé, gyorsabbá és fenntarthatóbbá tegye a divatcikkek adásvételét, miközben kellemes és megbízható élményt nyújt mind az eladók, mind a vásárlók számára. A Bellora segítségével a felhasználók könnyedén lépést tarthatnak a tudatos fogyasztás és a körforgásos gazdaság trendjével, miközben minimalizálják a felesleges hulladékot és pénzt takarítanak meg.

A Bellora lehetővé teszi a gyors és hatékony hirdetések kezelését, leegyszerűsíti az adásvételi folyamatot, és növeli a felhasználók elégedettségét. Az intelligens piactéri rendszernek köszönhetően az eladók könnyen kezelhetik hirdetéseiket, a vásárlók pedig egyszerűen böngészhetnek és találhatnak egyedi darabokat. Az automatizált folyamatok révén csökken a manuális munka, és gördülékenyebbé válik az egész tranzakció.

A platform felhasználóbarát, mobilról is tökéletesen használható kezelőfelületet kínál, amely lehetővé teszi a felhasználóknak, hogy bármikor, bárhonnan kezeljék hirdetéseiket, üzeneteiket és vásárlásaikat. Az élő frissítéseknek köszönhetően a hirdetések módosításai, új feltöltések és árváltozások azonnal láthatóvá válnak. A Bellora gyorsan használhatóvá válik, könnyen testreszabható és folyamatosan fejlődik a felhasználói igények alapján.

Kinek és miért?

Azért hoztuk létre a Bellorát, mert rengetegen szeretnék értékesíteni a már nem hordott ruháikat, cipőiket és kiegészítőiket, de hiányzik egy egyszerű, megbízható és modern platform erre Magyarországon. A Bellora mindazoknak szól, akik használt, jó állapotú divatcikkeket szeretnének eladni vagy vásárolni.

A Bellora elsősorban azoknak ajánlott, akik:

- Ki akarják pucolni a szekrényüket és pénzt csinálni a nem használt ruháikból
- Kedvező áron szeretnének minőségi, márkás darabokat vásárolni
- Tudatosan, fenntartható módon fogyasztanak és támogatják a körforgásos divatot

Az eladók számára a Bellora egyszerű és gyors lehetőséget nyújt arra, hogy hirdetéseket hozzanak létre, kezeljék az üzeneteket, és biztonságosan értékesítsenek. Így plusz bevételhez juthatnak, miközben helyet szabadítanak fel otthonukban.

A vásárlók számára kényelmes böngészést, részletes szűrőket, biztonságos fizetést és gyors ügyintézés nyújt. Egyedi, megfizethető és fenntartható módon juthatnak hozzá divatos ruhákhoz, cipőkhöz és kiegészítőkhöz.

A Bellora nemcsak egy egyszerű használt ruhás oldal, hanem egy modern, felhasználóbarát online marketplace, amely elősegíti a fenntartható divatot, segíti a közösséget és megkönnyíti a körforgásos fogyasztást. A platform használatával a

felhasználók pénzt takaríthatnak meg, csökkenthetik a környezeti terhelést, és hozzájárulhatnak egy tudatosabb divatiparhoz.

2. Fejlesztői dokumentáció

A fejlesztői dokumentáció átfogóan bemutatja azokat az eszközöket, amelyekkel a projekt készült, valamint részletezi a program telepítési lépéseit, futási környezetét, az adatbázis- és backend adatszerkezeteket. Emellett kiemel néhány kulcsfontosságú metódust a kódból, ismerteti a tesztelési folyamatot, és javasol további bővítési ötleteket a jövőbeli fejlesztésekhez.

Fejlesztőkörnyezet

A fejlesztőkörnyezet leírja azokat a szoftvereket, programokat és struktúrákat, amelyek a Bellora elkészítéséhez kellettek, és biztosítják annak zavartalan működését.

Használt IDE-k:

Visual Studio Code (VS Code)

A Visual Studio Code (VS Code) egy ingyenes, nyílt forráskódú, pehelysúlyú, de rendkívül hatékony kódszerkesztő, amit a Microsoft készített. Tökéletesen alkalmas JavaScript, React.js, Node.js és sok más nyelv projektekhez, így ideális full stack alkalmazások fejlesztéséhez. Az okos kódkiegészítés (IntelliSense), a natív Git-támogatás és a beépített terminál felgyorsítja a munkafolyamatot. Számtalan bővítmény (extension) érhető el hozzá, amelyekkel személyre szabható, például React Developer Tools vagy ESLint használatával.

A VS Code beépített hibakeresővel rendelkezik Node.js és böngészőalapú debuggoláshoz. A Live Share funkció real-time csapatmunkát tesz lehetővé, ami ideális kollaborációhoz. Felhasználóbarát interfésze és gyorsasága miatt a legnépszerűbb eszközök egyike, kompatibilis Windows, macOS és Linux rendszerekkel egyaránt.

Technikai Információk

Operációs Rendszer: Windows 10/11

Processzor: Intel Core i3 7100

RAM: 8GB

Storage: 1GB szabad tárhely

Internet

A fejlesztéshez a Windows 10 és 11 rendszerek a legmegfelelőbbek, mivel kiválóan támogatják a kortárs fejlesztői eszközöket és széles hardver-szoftver kompatibilitást

nyújtanak. Ezek a platformok egyszerűen kezelik a Node.js, MySQL és XAMPP telepítését, konfigurációját. Népszerű eszközök, mint a Visual Studio Code – amely natív támogatást ad JavaScripthez és React.js-hez –, valamint a WebStorm is zökkenőmentesen futnak rajtuk. A WSL (Windows Subsystem for Linux) révén Linux parancssori eszközök is elérhetőek Windows alatt, így minden fejlesztési fázisban stabil alapot biztosítanak.

Frontend

A kliensoldali felület építéséhez a következő technológiákat alkalmazzuk:

HTML – A HyperText Markup Language (HTML) képezi az app struktúrájának alapját, definiálja az oldal elemeit és a felhasználói interfészt. Friss szabványai biztosítják a böngésző-kompatibilitást, az akadálymentességet és a keresőoptimalizálást (SEO), miközben támogatják a karbantarthatóságot és reszponzív terveket.

CSS – A Cascading Style Sheets (CSS) felelős a vizuális formázásért, modern, reszponzív és vonzó megjelenést teremtve. Animációk, interakciók beépítésével gazdagítja a felhasználói élményt, míg CSS-preprocesszorok (pl. SASS, LESS) javítják a kód szervezését és teljesítményét.

React (v18+) – A React egy intuitív, nagy teljesítményű JavaScript library a dinamikus UI-k kezeléséhez. A 18+ verzió fejlett optimalizációkat és modern webes szabványokat kínál. Ökoszisztémája támogatja a React Router navigációt, Context API állapotkezelést, valamint Socket.io-client valós idejű kommunikációt. Komponensalapú felépítése skálázható, hatékony appok készítését teszi lehetővé.

Tailwind CSS (v3.4.13) – Utility-first CSS framework gyors, egyedi stílusokhoz külső fájlok nélkül. A 3.4.13 verzió optimalizált, konfigurálható, utility osztályaival felgyorsítja a munkát, miközben olvasható kódot eredményez. Ideális reszponzív, modern felületekhez.

NPM (v10.9.0) – A Node Package Manager gondoskodik a front-end függőségek (pl. Axios HTTP-kérésekhez, React Router) kezeléséről és telepítéséről, naprakészen tartva őket kompatibilitási gondok nélkül.

Backend

A szerveroldali logika és API-k kifejlesztéséhez ezeket a technológiákat használjuk:

JavaScript – Egységes nyelv a full stack fejlesztéshez, dinamikus, skálázható, Node.js-ben aszinkron műveletekkel kiemelkedő teljesítményt nyújtva a hagyományos szerverekkel szemben.

Node.js (v18+) – Gyors, eseményvezérelt JavaScript runtime szerveroldali appokhoz. A 18+ verzió stabil, V8-alapú, nagy terhelésű, real-time feladatokra optimalizált (pl. Socket.io WebSocket chathez).

Express.js (v4.21.1) – Minimalista Node.js framework REST API-k gyors építéséhez. A 4.21.1 verzió friss javításokkal kompatibilis a Node.js-sel, middleware-ekkel (pl. JWT autentikáció, bcrypt jelszóhash) bővíthető.

NPM (v10.9.0) – Backend csomagok (pl. Nodemon fejlesztői reloadhoz, Prisma ORM-hez, Multer fájlfeltöltéshez) kezeléséhez, egyszerű CLI-konfigurációval.

Adatbázis

Az adatok tárolásához és kezeléséhez MySQL-alapú megoldást alkalmazunk:

XAMPP Control Panel (v3.3.0) – All-in-one helyi szerverkörnyezet Apache-val és MySQL/MariaDB-val. Gyors setup-ot, adatbázis-tesztelést kínál éles környezet előtt.

MySQL (10.4.32-MariaDB) – Nyílt forráskódú, megbízható relationalis DBMS kiváló teljesítménnyel. A 10.4.32 verzió ACID-támogatással, indexeléssel kezeli nagy adatmennyiségeket és komplex query-ket, Prisma ORM-mel integrálva.

3. Adatszerkezetek

Adatbázis dokumentációja

A Bellora adatbázisa egy Vinted-szerű online piactér támogatására készült, amely termékek, ajánlatok, rendelések, üzenetek, fizetések, szállítások, értékelések, jelentések és felhasználói interakciók kezelésére szolgál.
Az alábbiakban ismertetjük az adatbázis struktúráját és a Prisma schema alapján létrehozott táblákat.

Táblák és oszlopok

1. categories (Kategóriák)

Tárolja a termékkategóriákat, támogatja a hierarchikus (szülő–gyermek) kapcsolatokat.

- `category_id` (int): Egyedi azonosító, auto-increment.
- `name` (varchar(100)): Kategória neve.
- `parent_id` (int?): Szülő kategória azonosítója (opcionális).

2. favorites (Kedvencek)

Tárolja a felhasználók által kedvelt termékeket.

- `user_id` (int): Felhasználó azonosítója.
- `item_id` (int): Termék azonosítója.

3. itemimages (Termékképek)

Tárolja a termékekhez tartozó képeket.

- `image_id` (int): Egyedi azonosító, auto-increment.
- `item_id` (int): Termék azonosítója.
- `image_url` (varchar(500)): Kép URL-je.
- `is_primary` (boolean): Fő kép-e (alapértelmezett: false).
- `display_order` (int?): Megjelenítési sorrend.
- `uploaded_at` (datetime): Feltöltés időpontja.

4. items (Termékek)

Tárolja az eladóként feladott termékeket.

- `item_id` (int): Egyedi azonosító, auto-increment.
- `user_id` (int): Eladó felhasználó azonosítója.
- `category_id` (int): Kategória azonosítója.
- `title` (varchar(150)): Termék címe.
- `description` (text?): Termék leírása.
- `price` (decimal(15,2)): Termék ára.
- `status` (enum: available/sold/reserved): Termék állapota.
- `created_at` (datetime): Létrehozás időpontja.

5. messages (Üzenetek)

Tárolja a vevő–eladó közötti chat üzeneteket.

- `message_id` (int): Egyedi azonosító, auto-increment.
- `sender_id` (int): Küldő felhasználó azonosítója.
- `receiver_id` (int): Címzett felhasználó azonosítója.
- `content` (text): Üzenet tartalma.
- `sent_at` (datetime): Küldés időpontja.
- `offer_id` (int?): Kapcsolódó ajánlat azonosítója (opcionális).

6. offers (Ajánlatok)

Tárolja a termékekre tett ajánlatokat.

- `offer_id` (int): Egyedi azonosító, auto-increment.
- `item_id` (int): Termék azonosítója.
- `buyer_id` (int): Ajánlattevő (vevő).
- `seller_id` (int): Eladó.
- `offer_price` (decimal(15,2)): Ajánlott ár.
- `status` (enum: pending/accepted/rejected/counter_offered/cancelled): Ajánlat állapota.
- `counter_price` (decimal(15,2)?): Ellenajánlat ára.
- `created_at` (datetime): Létrehozás időpontja.
- `updated_at` (datetime): Frissítés időpontja.

7. orders (Rendelések)

Tárolja a leadott és feldolgozott rendeléseket.

- `order_id` (int): Egyedi azonosító, auto-increment.
- `buyer_id` (int): Vevő azonosítója.
- `seller_id` (int): Eladó azonosítója.
- `item_id` (int): Termék azonosítója.
- `order_date` (datetime): Rendelés időpontja.
- `status` (enum: pending/completed/cancelled): Rendelés állapota.

8. payments (Fizetések)

Tárolja a rendelésekhez tartozó fizetéseket.

- `payment_id` (int): Egyedi azonosító, auto-increment.
- `order_id` (int): Rendelés azonosítója (egyedi).
- `amount` (decimal(10,2)): Fizetendő összeg.
- `method` (varchar(50)?): Fizetési mód.
- `status` (enum: pending/completed/failed): Fizetés állapota.
- `created_at` (datetime): Létrehozás időpontja.

9. reviews (Értékelések)

Tárolja a felhasználók közötti értékeléseket.

- `review_id` (int): Egyedi azonosító, auto-increment.
- `reviewer_id` (int): Értékelő felhasználó.
- `reviewed_user_id` (int): Értékelt felhasználó.
- `rating` (int): Csillagok száma.
- `comment` (text?): Szöveges értékelés.
- `created_at` (datetime): Létrehozás időpontja.

10. shipping (Szállítások)

Tárolja a rendelések szállítási adatait.

- `shipping_id` (int): Egyedi azonosító, auto-increment.
- `order_id` (int): Rendelés azonosítója (egyedi).
- `address` (text): Szállítási cím.
- `carrier` (varchar(50)?): Szállítmányozó.
- `tracking_number` (varchar(100)?): Követési szám.
- `status` (enum: processing/shipped/delivered/cancelled): Szállítás állapota.

11. reports (Jelentések)

Tárolja a felhasználók által beküldött jelentéseket problémás hirdetésekről.

- `report_id` (int): Egyedi azonosító, auto-increment.
- `item_id` (int): Jelentett termék.
- `reporter_id` (int): Jelentést beküldő felhasználó.
- `reason` (text): Jelentés oka.
- `status` (enum: pending/reviewed/dismissed): Jelentés állapota.
- `created_at` (datetime): Létrehozás időpontja.

12. users (Felhasználók)

Tárolja a regisztrált felhasználókat.

- `user_id` (int): Egyedi azonosító, auto-increment.
- `username` (varchar(50)): Felhasználónév (egyedi).
- `email` (varchar(100)): E-mail cím (egyedi).
- `password_hash` (text): Jelszó hash.
- `full_name` (varchar(100)?): Teljes név.
- `phone` (varchar(20)?): Telefonszám.
- `address` (varchar(255)?): Cím.
- `city` (varchar(100)?): Város.
- `postal_code` (varchar(10)?): Irányítószám.
- `join_date` (datetime): Regisztráció időpontja.
- `daily_offers_count` (int?): Napi ajánlatok száma.
- `last_offer_reset` (date?): Utolsó ajánlat-reset dátuma.
- `profile_image` (varchar(255)?): Profilkép URL.
- `role` (enum: user/admin): Felhasználói szerepkör.

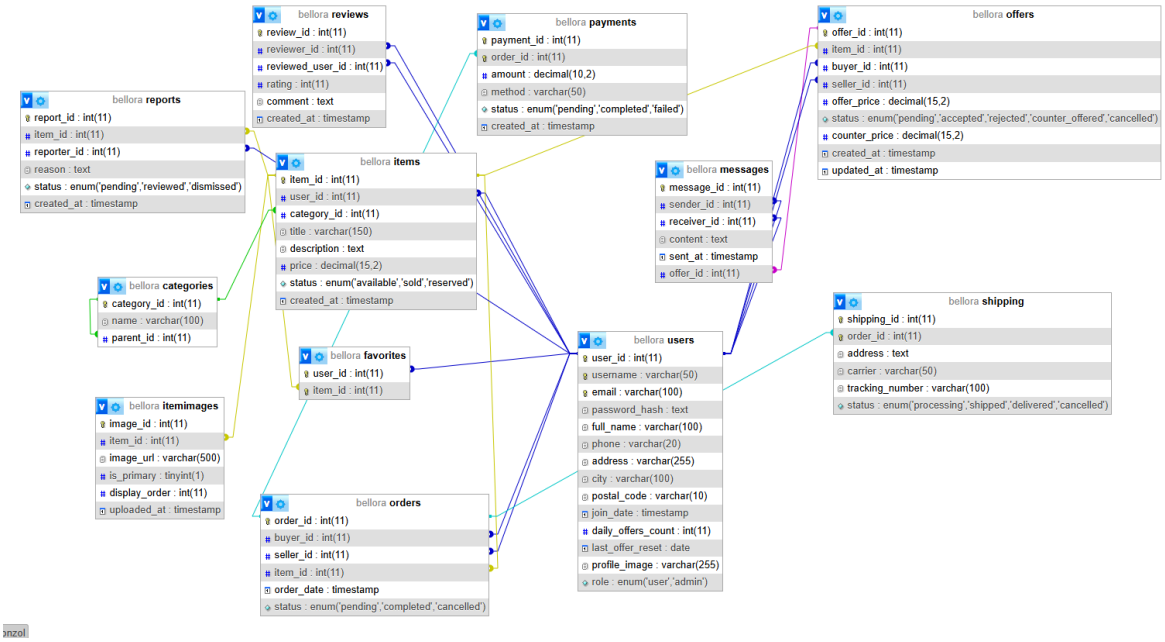
Kapcsolatok az adatbázisban

- Az `items` tábla a `categories`-re és `users`-re hivatkozik (eladó).
- A `favorites`, `offers`, `orders`, `reports` táblák az `items`-hez és `users`-höz kapcsolódnak.
- A `messages` a `users` között hoz létre kapcsolatot, opcionálisan `offers` rekordhoz kötve.
- A `payments` és `shipping` egyedi kapcsolatot tart fenn az `orders` táblával (1:1).
- A `reviews` két `users` között valósít meg értékelést.
- A `itemimages` a termékekhez tartozik (1:N).
- A `categories` hierarchikus önkapcsolata `parent_id` alapján.

Ez a Prisma-alapú adatbázis-struktúra biztosítja a Bellora piactér hatékony működését, támogatva a termékkezelést, ajánlat-ajánlattevést, üzenetváltást, rendeléseket, fizetéseket, szállításokat, értékeléseket és jelentéseket.

ER (Entity Relationship) Diagramm

Ez az ER-diagram a Bellora online piactér rendszert modellezi. A Users entitás kezeli a regisztrált felhasználókat profiladatokkal, míg az Items a termékeket kategóriákba sorolva, képekkel és státuszokkal. Az Offers és Orders követik az ajánlatokat és vásárlásokat vevő-eladó kapcsolatokkal. A Messages támogatja a chatet, a Payments és Shipping a tranzakciókat/szállításokat, a Reviews az értékeléseket, a Favorites a kedvenceket kezeli. A hierarchikus Categories biztosítja a termékböngészést. Az adatmodell átláthatóan támogatja a piactéri folyamatokat: eladást, licitálást, kommunikációt és logisztikát.



Egyedi algoritmusok

Ebben a fejezetben bemutatunk néhány kulcsfontosságú, egyedi implementált algoritmust. Ezek integrálva vannak a Bellora applikációba, a programozási legjobb gyakorlatok szerint tervezve, implementálva. Alább kódrészletek és rövid magyarázatok találhatók a működésükről és céljukról.

4. Tesztek

Backend: A backend rendszerhez készült automatizált tesztek mind sikeresen lefutottak. Összesen 7 tesztcsomag és 18 teszt került végrehajtásra, amelyek lefedik a fő funkciókat: felhasználók, hitelesítés, admin, termékek, kategóriák, rendelések és jelentések. A tesztek során néhány figyelmeztetés és konzolüzenet jelent meg (dotenv, localstorage warning), de ezek nem befolyásolták a működést. Az auth tesztekben megjelenő hibák a hibakezelés teszteléséből adódnak, és a rendszer helyesen reagált ezekre. A tesztek eredménye alapján a backend stabil, a fő funkciók megfelelően működnek.

```
Test Suites: 7 passed, 7 total
Tests:       18 passed, 18 total
Snapshots:   0 total
Time:        6.619 s
Ran all test suites.
```

Frontend: A frontend end-to-end tesztek 14 sikeres futással igazolták, hogy a főoldal, navigáció, hirdetés-részletek, regisztráció, bejelentkezés, admin és védett oldalak minden kulcsfunkciója megfelelően működik és helyesen kezeli a jogosultságokat, hibákat és felhasználói útvonalakat.

```
Running 14 tests using 2 workers
  14 passed (13.7s)

To open last HTML report run:

  npx playwright show-report
```

```
PASS teszt/categories.test.js
  ● Console

    console.log
      [dotenv@17.2.3] injecting env (0) from .env -- tip: ⚙️ specify custom .env file path with { path: '/custom/path/.env' }

      at _log (node_modules/dotenv/lib/main.js:142:11)

    console.log
      [dotenv@17.2.3] injecting env (0) from .env -- tip: ⚙️ override existing env vars with { override: true }

      at _log (node_modules/dotenv/lib/main.js:142:11)

    console.log
      Adatbázis kapcsolódva

      at log (server.js:41:11)

PASS teszt/reports.test.js
  ● Console

    console.log
      [dotenv@17.2.3] injecting env (0) from .env -- tip: 🔑 add access controls to secrets: https://dotenvx.com/ops

      at _log (node_modules/dotenv/lib/main.js:142:11)

    console.log
      [dotenv@17.2.3] injecting env (0) from .env -- tip: ⚙️ suppress all logs with { quiet: true }

      at _log (node_modules/dotenv/lib/main.js:142:11)

    console.log
      Adatbázis kapcsolódva

      at log (server.js:41:11)
```

```
PASS teszt/auth.test.js
  ● Console

    console.log
      [dotenv@17.2.3] injecting env (0) from .env -- tip: 🔐 encrypt with Dotenvx: https://dotenvx.com

      at _log (node_modules/dotenv/lib/main.js:142:11)

    console.log
      [dotenv@17.2.3] injecting env (0) from .env -- tip: ✅ audit secrets and track compliance: https://dotenvx.com/ops

      at _log (node_modules/dotenv/lib/main.js:142:11)

    console.log
      Adatbázis kapcsolódva

      at log (server.js:41:11)
```

```
PASS teszt/users.test.js
  ● Console

    console.log
      [dotenv@17.2.3] injecting env (0) from .env -- tip: ⚙️ override existing env vars with { override: true }

      at _log (node_modules/dotenv/lib/main.js:142:11)

    console.log
      [dotenv@17.2.3] injecting env (0) from .env -- tip: ⚙️ specify custom .env file path with { path: '/custom/path/.env' }

      at _log (node_modules/dotenv/lib/main.js:142:11)

    console.log
      Adatbázis kapcsolódva

      at log (server.js:41:11)
```

```
PASS  teszt/admin.test.js
  ● Console

    console.log
      [dotenv@17.2.3] injecting env (0) from .env -- tip: ⚙️ suppress all logs with { quiet: true }

      at _log (node_modules/dotenv/lib/main.js:142:11)

    console.log
      [dotenv@17.2.3] injecting env (0) from .env -- tip: 🔐 encrypt with Dotenvx: https://dotenvx.com

      at _log (node_modules/dotenv/lib/main.js:142:11)

    console.log
      Adatbázis kapcsolódva

      at log (server.js:41:11)

PASS  teszt/items.test.js
  ● Console

    console.log
      [dotenv@17.2.3] injecting env (0) from .env -- tip: ✅ audit secrets and track compliance: https://dotenvx.com/ops

      at _log (node_modules/dotenv/lib/main.js:142:11)

    console.log
      [dotenv@17.2.3] injecting env (0) from .env -- tip: ⚙️ write to custom object with { processEnv: myObject }

      at _log (node_modules/dotenv/lib/main.js:142:11)

    console.log
      Adatbázis kapcsolódva

      at log (server.js:41:11)
```

```
PASS  teszt/orders.test.js
  ● Console

    console.log
      [dotenv@17.2.3] injecting env (0) from .env -- tip: ⚙️ specify custom .env file path with { path: '/custom/path/.env' }

      at _log (node_modules/dotenv/lib/main.js:142:11)

    console.log
      [dotenv@17.2.3] injecting env (0) from .env -- tip: 🔐 prevent committing .env to code: https://dotenvx.com/precommit

      at _log (node_modules/dotenv/lib/main.js:142:11)

    console.log
      Adatbázis kapcsolódva

      at log (server.js:41:11)
```

Search tests | All 14 | Passed 14 | Failed 0 | Flaky 0 | Skipped 0

Project: chromium — 2026. 04. 29. 21:10:02 Total time: 8.4s

admin.spec.js

- Admin oldal tesztelése › Admin oldal átirányít bejelentkezés nélkül chromium — 2.7s
  admin.spec.js:4

auth.spec.js

- Bejelentkezési funkció tesztelése (Auth) › Helytelen adatokkal hibaüzenetet kell mutatnia chromium — 2.2s
  auth.spec.js:4

home.spec.js

- Főoldal tesztelése › A főoldal betöltődik és megjeleníti a Bellora logót chromium — 2.0s
  home.spec.js:4
- Főoldal tesztelése › A hirdetések megjelennek a főoldalon chromium — 3.7s
  home.spec.js:9

itemDetail.spec.js

- Hirdetés részletek tesztelése › Nem létező hirdetés oldal betöltődik (loading állapot) chromium — 3.7s
  itemDetail.spec.js:4
- Hirdetés részletek tesztelése › Hirdetés oldalon megjelenik a navbar chromium — 2.0s
  itemDetail.spec.js:12

navigation.spec.js

- Navigáció tesztelése › A navbar megjelenik és tartalmazza a fő linkeket chromium — 2.0s
  navigation.spec.js:4
- Navigáció tesztelése › A bejelentkezés oldalra navigálás működik chromium — 2.0s
  navigation.spec.js:10
- Navigáció tesztelése › A regisztráció oldalra navigálás működik chromium — 2.0s
  navigation.spec.js:17

protected.spec.js

- Védett oldalak tesztelése › Profil oldal átirányít bejelentkezés nélkül chromium — 2.7s
  protected.spec.js:4
- Védett oldalak tesztelése › Új hirdetés oldal átirányít bejelentkezés nélkül chromium — 2.8s
  protected.spec.js:10
- Védett oldalak tesztelése › Kedvencek oldal átirányít bejelentkezés nélkül chromium — 2.7s
  protected.spec.js:16

register.spec.js

- Regisztrációs oldal tesztelése › A regisztrációs form megjelenik az összes mezővel chromium — 308ms
  register.spec.js:4
- Regisztrációs oldal tesztelése › Üres form beküldése nem navigál el chromium — 361ms
  register.spec.js:12

Teszt fiókok:

User:

Email-cím: teszt@teszt.teszt

Jelszó:teszt123

Admin:

Email-cím: admin@admin.admin

Jelszó: admin123

Fejlesztési lehetőségek:

FIZETÉSI RENDSZER INTEGRÁCIÓ

- Stripe vagy PayPal fizetési kapu integrálása
- Online fizetés támogatása (bankkártya, PayPal)
- Fizetési visszaigazolás és számla generálás

EMAIL ÉRTESÍTÉSEK

- Regisztrációs visszaigazolás email
- Rendelés visszaigazolás email
- Jelszó visszaállítás email
- Új ajánlat értesítés email

Nodemailer vagy SendGrid integráció

Backend felépítése

authController.js

A hitelesítési logika kezeli a felhasználói regisztrációt, bejelentkezést és profillekérdezést Prisma ORM-mel, bcrypt jelszóhash-el és JWT tokenekkel.

A register függvény új felhasználót hoz létre: duplikációt ellenőriz (email/username), bcrypt-tel hasheli a jelszót (10 salt round), Prisma-val menti az adatbázisba, majd JWT tokent generál azonosításhoz, amit a kliens tárolhat

```
export const register = async (req, res) => {
  try {
    const { username, email, password } = req.body;

    const existing = await prisma.users.findFirst({
      where: {
        OR: [{ email }, { username }]
      }
    });
    if (existing) {
      return res.status(400).json({ message: 'A felhasználó már létezik' });
    }

    const password_hash = await bcrypt.hash(password, 10);
    const user = await prisma.users.create({
      data: { username, email, password_hash }
    });

    const token = jwt.sign({ user_id: user.user_id, username, email }, process.env.JWT_SECRET);

    res.json({
      message: 'Sikeres regisztráció',
      token,
      user: { user_id: user.user_id, username, email }
    });
  } catch (error) {
    console.error(error);
    res.status(500).json({ message: 'Szerver hiba' });
  }
};
```

```
export const login = async (req, res) => {
  try {
    const { email, password } = req.body;

    const user = await prisma.users.findUnique({ where: { email } });
    if (!user) {
      return res.status(401).json({ message: 'Hibás email vagy jelszó' });
    }

    const valid = await bcrypt.compare(password, user.password_hash);
    if (!valid) {
      return res.status(401).json({ message: 'Hibás email vagy jelszó' });
    }

    const token = jwt.sign({ user_id: user.user_id, username: user.username, email: user.email }, process.env.JWT_SECRET);

    res.json({
      message: 'Sikeres bejelentkezés',
      token,
      user: { user_id: user.user_id, username: user.username, email: user.email, profile_image: user.profile_image }
    });
  } catch (error) {
    console.error(error);
    res.status(500).json({ message: 'Szerver hiba' });
  }
};
```

A login függvény email alapján keresi a felhasználót, bcrypt-tel validálja a jelszót, majd JWT tokent ad vissza profilinfókkal (pl. profilképpel). Ez lehetővé teszi a stateless autentikációt API-kban.

```
export const getCurrentUser = async (req, res) => {
  try {
    const user = await prisma.users.findUnique({
      where: { user_id: req.user.user_id },
      select: {
        user_id: true,
        username: true,
        email: true,
        full_name: true,
        phone: true,
        address: true,
        city: true,
        postal_code: true,
        profile_image: true,
        join_date: true
      }
    });
    res.json({ user });
  } catch (error) {
    console.error(error);
    res.status(500).json({ message: 'Szerver hiba' });
  }
};
```

A getCurrentUser middleware-ből (JWT) veszi a user_id-t, Prisma select-tel biztonságosan lekéri a profiladatokat (pl. cím, telefon), így támogatja a felhasználói dashboard-ot anélkül, hogy érzékeny adatokat (pl. jelszó) exponálna.

A categoryController.js function lekéri az összes fő kategóriát (ahol parent_id null) a Prisma ORM segítségével. A kategóriákat JSON formátumban küldi vissza a kliensnek. Ez a végpont tipikusan a webshop vagy tartalomkezelő rendszer kategórianavigációjának betöltéséhez használatos.

```
import prisma from '../config/db.js';

export const getAllCategories = async (req, res) => {
    try {
        const categories = await prisma.categories.findMany({
            where: { parent_id: null }
        });
        res.json({ categories });
    } catch (error) {
        console.error(error);
        res.status(500).json({ message: 'Szerver hiba' });
    }
};
```

```
export const getFavorites = async (req, res) => {
  try {
    const favoriteRecords = await prisma.favorites.findMany({
      where: { user_id: req.user.user_id },
      include: {
        items: {
          include: {
            users: {
              select: { username: true, profile_image: true }
            },
            itemimages: {
              orderBy: { display_order: 'asc' },
              take: 1,
              select: { image_url: true }
            }
          }
        }
      }
    });

    const favorites = favoriteRecords.map(f => ({
      ...f.items,
      seller_name: f.items.users.username,
      seller_image: f.items.users.profile_image,
      image_url: f.items.itemimages[0]?.image_url || null,
      users: undefined,
      itemimages: undefined
    }));

    res.json({ favorites });
  } catch (error) {
    console.error(error);
    res.status(500).json({ message: 'Szerver hiba' });
  }
};
```

A getFavorites function lekéri a bejelentkezett felhasználó kedvenc termékeit. Prisma findMany lekérdezéssel betölti a kedvencekhez tartozó itemeket, az eladó adatait (username, profile_image) és az item első képét. Az eredményt egy jól strukturált objektummá alakítja (seller_name, seller_image, image_url stb.), majd JSON formátumban visszaküldi. Ez a végpont a felhasználói profil „Kedvenceim” oldalának feltöltésére szolgál.

Az addFavorite function függvény lehetővé teszi, hogy a bejelentkezett felhasználó egy terméket a kedvencei közé adjon. Prisma upsert metódusát használja, amely automatikusan kezeli, hogy az adott user_id + item_id kombináció már létezik-e. Ha még nem volt kedvencként elmentve, létrehozza az új rekordot; ha már létezett, akkor nem végez módosítást. Sikeres hozzáadás után egy megerősítő üzenetet küld vissza: „Hozzáadva a kedvencekhez”.

```
export const addFavorite = async (req, res) => {
  try {
    const item_id = parseInt(req.body.item_id);
    await prisma.favorites.upsert({
      where: {
        user_id_item_id: { user_id: req.user.user_id, item_id
      },
      update: {},
      create: { user_id: req.user.user_id, item_id }
    });
    res.json({ message: 'Hozzáadva a kedvencekhez' });
  } catch (error) {
    console.error(error);
    res.status(500).json({ message: 'Szerver hiba' });
  }
};
```

Az removeFavorite function függvény eltávolítja a megadott terméket a bejelentkezett felhasználó kedvencei közül.
A prisma.favorites.delete() metódust használja, ahol a törlési feltétel a user_id és az item_id kombinációja (összetett kulcs).
Az item_id-t a kérés URL paraméteréből (req.params.itemId) olvassa ki.
Sikeres művelet esetén a válasz: „Eltávolítva a kedvencekből”.

```
export const removeFavorite = async (req, res) => {
  try {
    await prisma.favorites.delete({
      where: {
        user_id_item_id: {
          user_id: req.user.user_id,
          item_id: parseInt(req.params.itemId)
        }
      }
    });
    res.json({ message: 'Eltávolítva a kedvencekből' });
  } catch (error) {
    console.error(error);
    res.status(500).json({ message: 'Szerver hiba' });
  }
};
```

A getAllItems function az adatbázisból minden olyan terméket lekér, amelynek az állapota „available”. A lekérdezést a prisma.items.findMany() végzi, amely a kapcsolódó felhasználótól csak a felhasználónevet, az itemimages táblából az elsődleges kép URL-jét, valamint a kedvencek számát is csatolja. Az eredményeket a created_at mező alapján csökkenő sorrendben rendezi. A kapott listát a függvény tovább alakítja: minden elemhez hozzáadja az eladó nevét, a kép URL-jét és a kedvencek számát, miközben néhány mezőt kifejezetten undefined értékre állít. A végső, átalakított adatstruktúrát JSON formában küldi vissza a kliensnek.

```
export const getAllItems = async (req, res) => {
  try {
    const items = await prisma.items.findMany({
      where: { status: 'available' },
      include: {
        users: { select: { username: true } },
        itemimages: {
          where: { is_primary: true },
          take: 1,
          select: { image_url: true }
        },
        _count: { select: { favorites: true } }
      },
      orderBy: { created_at: 'desc' }
    });

    const result = items.map(item => ({
      ...item,
      seller_name: item.users.username,
      image_url: item.itemimages[0]?.image_url || null,
      favorites_count: item._count.favorites,
      users: undefined,
      itemimages: undefined,
      _count: undefined
    }));

    res.json({ items: result });
  } catch (error) {
    console.error(error);
    res.status(500).json({ message: 'Szerver hiba' });
  }
};
```

A createItem function egy új hirdetést hoz létre az adatbázisban. Először ellenőrzi, hogy az ár meg van-e adva és nagyobb-e nullánál; ha nem, 400-as hibát küld vissza. Ezután a Prisma items.create() metódusával elmenti a hirdetést a bejelentkezett felhasználóhoz kapcsolva, a kategória-ID-t

```
export const createItem = async (req, res) => {
  try {
    const { title, description, price, category_id } = req.body;

    if (!price || parseFloat(price) <= 0) {
      return res.status(400).json({ message: 'Az árnak nullánál nagyobbnak kell lennie' });
    }

    const item = await prisma.items.create({
      data: {
        user_id: req.user.user_id,
        category_id: category_id ? parseInt(category_id) : 1,
        title,
        description,
        price: parseFloat(price)
      }
    });
    res.json({ message: 'Hirdetés létrehozva', item_id: item.item_id });
  } catch (error) {
    console.error(error);
    res.status(500).json({ message: 'Szerver hiba' });
  }
};
```

számmá alakítva, az árat pedig lebegőpontos értékként tárolva. Siker esetén visszaküldi a „Hirdetés létrehozva” üzenetet és az új elem azonosítóját.

A updateItem function egy meglévő hirdetés módosítását végzi. Először kiolvassa a kérésből az új adatokat, majd ellenőrzi, hogy az ár meg van-e adva és nagyobb-e nullánál; ha nem, 400-as hibát küld vissza. Ezután lekéri az adott hirdetést az adatbázisból, és ha nem található, 404-es hibát ad. Ha a hirdetés nem a bejelentkezett felhasználóhoz tartozik, 403-as hibával jelzi a jogosultság hiányát. Minden feltétel teljesülése esetén a Prisma items.update() metódusával frissíti a hirdetés adatait, majd visszaküldi a „Hirdetés frissítve” üzenetet.

```
export const updateItem = async (req, res) => {
  try {
    const { title, description, price, category_id, status } = req.body;
    const id = parseInt(req.params.id);

    if (!price || parseFloat(price) <= 0) {
      return res.status(400).json({ message: 'Az árnak nullánál nagyobbnak kell lennie' });
    }

    const item = await prisma.items.findUnique({
      where: { item_id: id },
      select: { user_id: true }
    });

    if (!item) {
      return res.status(404).json({ message: 'Hirdetés nem található' });
    }

    if (item.user_id !== req.user.user_id) {
      return res.status(403).json({ message: 'Nincs jogosultságod szerkeszteni ezt a hirdetést' });
    }

    await prisma.items.update({
      where: { item_id: id },
      data: {
        title,
        description,
        price: parseFloat(price),
        category_id: parseInt(category_id),
        status
      }
    });

    res.json({ message: 'Hirdetés frissítve' });
  } catch (error) {
    console.error(error);
    res.status(500).json({ message: 'Szerver hiba' });
  }
};
```

A deleteItem function egy hirdetés törlését végzi. Először kiolvassa az URL-ből az azonosítót, majd lekéri az adatbázisból a hirdetés tulajdonosának azonosítóját. Ha a hirdetés nem létezik, 404-es választ ad. Ha a bejelentkezett felhasználó nem egyezik a hirdetés tulajdonosával, 403-as hibával jelzi a jogosultság hiányát. Amennyiben minden feltétel teljesül, a Prisma items.delete() metódusával törli a

```
export const deleteItem = async (req, res) => {
  try {
    const id = parseInt(req.params.id);

    const item = await prisma.items.findUnique({
      where: { item_id: id },
      select: { user_id: true }
    });

    if (!item) {
      return res.status(404).json({ message: 'Hirdetés nem található' });
    }

    if (item.user_id !== req.user.user_id) {
      return res.status(403).json({ message: 'Nincs jogosultságod törölni ezt a hirdetést' });
    }

    // Prisma cascades handle itemimages and favorites deletion
    await prisma.items.delete({ where: { item_id: id } });

    res.json({ message: 'Hirdetés törölve' });
  } catch (error) {
    console.error(error);
    res.status(500).json({ message: 'Szerver hiba' });
  }
};
```

hirdetést, és visszaküldi a „Hirdetés törölve” üzenetet.

A *getMessages* function két felhasználó közötti üzeneteket kér le az adatbázisból. A bejelentkezett felhasználó azonosítóját a tokenből, a másik fél azonosítóját az URL-paraméterből olvassa ki. A Prisma

```
export const getMessages = async (req, res) => {
  try {
    const userId = req.user.user_id;
    const otherUserId = parseInt(req.params.userId);

    const messages = await prisma.messages.findMany({
      where: {
        OR: [
          { sender_id: userId, receiver_id: otherUserId },
          { sender_id: otherUserId, receiver_id: userId }
        ]
      },
      include: {
        users_messages_sender_idTousers: { select: { username: true } },
        users_messages_receiver_idTousers: { select: { username: true } },
        offers: {
          select: {
            status: true,
            offer_price: true,
            counter_price: true,
            items: {
              select: { item_id: true, title: true, price: true, itemimages:
                { orderBy: [{ is_primary: 'desc' }, { display_order: 'asc' }],
                take: 1, select: { image_url: true } } }
            }
          }
        }
      },
      orderBy: { sent_at: 'asc' }
    });

    const result = messages.map(m => ({
      ...m,
      sender_name: m.users_messages_sender_idTousers.username,
      receiver_name: m.users_messages_receiver_idTousers.username,
      offer_status: m.offers?.status || null,
      offer_price: m.offers?.offer_price || null,
      counter_price: m.offers?.counter_price || null,
      item_id: m.offers?.items?.item_id || null,
      item_title: m.offers?.items?.title || null,
      item_price: m.offers?.items?.price || null,
      item_image: m.offers?.items?.itemimages?.[0]?.image_url || null,
      users_messages_sender_idTousers: undefined,
      users_messages_receiver_idTousers: undefined,
      offers: undefined
    }));

    res.json({ messages: result });
  } catch (error) {
    console.error(error);
    res.status(500).json({ message: 'Szerver hiba' });
  }
};
```

`messages.findMany()` metódusával minden olyan üzenetet lekér, ahol a két fél bármelyik irányban szerepel feladóként és címzettként. A lekérdezés tartalmazza a feladó és a címzett felhasználónevét, valamint – ha létezik – a kapcsolódó ajánlat adatait,

például az ajánlat státuszát, az ajánlott és ellenajánlott árat, illetve az ajánlathoz tartozó termék alapadatait és elsődleges képét. Az üzenetek időrendben, a `sent_at` mező alapján növekvő sorrendben érkeznek vissza.

A *sendMessage* function egy új üzenet elküldéséért felel. A függvény a kérés törzséből kiolvassa a címzett azonosítóját és az üzenet tartalmát, majd a Prisma

```
export const sendMessage = async (req, res) => {
  try {
    const receiver_id = parseInt(req.body.receiver_id);
    const { content } = req.body;
    await prisma.messages.create({
      data: {
        sender_id: req.user.user_id,
        receiver_id,
        content
      }
    });
    res.json({ message: 'Üzenet elküldve' });
  } catch (error) {
    console.error(error);
    res.status(500).json({ message: 'Szerver hiba' });
  }
};
```

`messages.create()` metódusával létrehoz egy új rekordot, ahol a feladó a bejelentkezett felhasználó, a címzett pedig a megadott `receiver_id`. Sikeres mentés után visszaküldi az „Üzenet elküldve” üzenetet.

A *getConversations* function a felhasználó összes beszélgetését gyűjti össze és rendezi. Először lekéri az adatbázisból az összes olyan üzenetet, ahol a bejelentkezett felhasználó feladóként vagy címzettként szerepel, és ezekhez csatolja mindkét fél felhasználónevét. Az üzeneteket időrendben, a legutóbbiaktól kezdve kapja meg. Ezután a függvény végigmegy az összes üzeneten, és minden üzenetnél meghatározza, ki a másik fél. Ha az adott másik felhasználóhoz még nincs beszélgetés rögzítve, létrehoz egy új bejegyzést, amely tartalmazza a másik fél azonosítóját, felhasználónevét, valamint a legutóbbi üzenet tartalmát és időpontját. A végén a beszélgetések listáját JSON formában küldi vissza.

```
export const getConversations = async (req, res) => {
  try {
    const userId = req.user.user_id;

    // Get all messages involving the user
    const allMessages = await prisma.messages.findMany({
      where: {
        OR: [
          { sender_id: userId },
          { receiver_id: userId }
        ]
      },
      include: {
        users_messages_sender_idTousers: { select: { user_id: true, username: true } },
        users_messages_receiver_idTousers: { select: { user_id: true, username: true } }
      },
      orderBy: { sent_at: 'desc' }
    });

    // Group by other user to get unique conversations
    const conversationMap = new Map();
    for (const msg of allMessages) {
      const otherUserId = msg.sender_id === userId ? msg.receiver_id : msg.sender_id;
      if (!conversationMap.has(otherUserId)) {
        const otherUser = msg.sender_id === userId
          ? msg.users_messages_receiver_idTousers
          : msg.users_messages_sender_idTousers;
        conversationMap.set(otherUserId, {
          other_user_id: otherUserId,
          other_user_name: otherUser.username,
          last_message: msg.content,
          last_message_time: msg.sent_at
        });
      }
    }

    const conversations = Array.from(conversationMap.values());
    res.json({ conversations });
  } catch (error) {
    console.error(error);
    res.status(500).json({ message: 'Szerver hiba' });
  }
};
```

```
export const getNotifications = async (req, res) => {
  try {
    const userId = req.user.user_id;

    const favoritesRaw = await prisma.favorites.findMany({
      where: {
        items: { user_id: userId }
      },
      include: {
        users: { select: { username: true, profile_image: true } },
        items: { select: { item_id: true, title: true } }
      },
      take: 20,
      orderBy: { item_id: 'desc' }
    });

    const favorites = favoritesRaw.map(f => ({
      favoriter_id: f.user_id,
      favoriter_name: f.users.username,
      favoriter_image: f.users.profile_image,
      item_id: f.items.item_id,
      item_title: f.items.title,
      created_at: new Date(),
      type: 'favorite'
    }));

    const ordersRaw = await prisma.orders.findMany({
      where: { seller_id: userId },
      include: {
        users_orders_buyer_idTousers: { select: { username: true, profile_image: true } },
        items: { select: { item_id: true, title: true } }
      },
      orderBy: { order_date: 'desc' },
      take: 20
    });
    const orders = ordersRaw.map(o => ({
      order_id: o.order_id,
      buyer_id: o.buyer_id,
      buyer_name: o.users_orders_buyer_idTousers.username,
      buyer_image: o.users_orders_buyer_idTousers.profile_image,
      item_id: o.items.item_id,
      item_title: o.items.title,
      created_at: o.order_date,
      order_status: o.status,
      type: 'order'
    }));

    const offersRaw = await prisma.offers.findMany({
      where: { seller_id: userId, status: 'pending' },
      include: {
        users_offers_buyer_idTousers: { select: { username: true, profile_image: true } },
        items: { select: { item_id: true, title: true } }
      },
      orderBy: { created_at: 'desc' },
      take: 20
    });

    const offers = offersRaw.map(o => ({
      offer_id: o.offer_id,
      buyer_id: o.buyer_id,
      buyer_name: o.users_offers_buyer_idTousers.username,
      buyer_image: o.users_offers_buyer_idTousers.profile_image,
      item_id: o.items.item_id,
      item_title: o.items.title,
      offer_price: o.offer_price,
      offer_status: o.status,
      created_at: o.created_at,
      type: 'offer'
    }));

    const allNotifications = [...favorites, ...orders, ...offers].sort((a, b) => {
      return new Date(b.created_at) - new Date(a.created_at);
    });
    res.json({
      notifications: allNotifications.slice(0, 20),
      count: allNotifications.length
    });
  } catch (error) {
    console.error(error);
    res.status(500).json({ message: 'Szerver hiba' });
  }
};
```

A getNotifications function a bejelentkezett felhasználó értesítéseit gyűjti össze három különböző forrásból: kedvencekből, rendelésekből és ajánlatokból. Először lekéri azokat a kedvenceket, ahol a felhasználó hirdetését mások megjelölték kedvencnek, majd ezeket egy egységes szerkezetű objektummá alakítja, amely tartalmazza a kedvelő felhasználó adatait és a hirdetés alapinformációit. Ezután lekéri a felhasználó eladóként kapott rendeléseit, és ugyanígy egységes formátumba rendezi őket, beleértve a vevő adatait, a hirdetés címét és a rendelés állapotát. Harmadik lépésként összegyűjti a felhasználó eladóként kapott, még függőben lévő ajánlatait, majd ezekből is értesítési objektumokat készít, amelyek tartalmazzák az ajánlat adatait és a kapcsolódó terméket. A három értesítéstípus egyetlen listába kerül, amelyet a létrehozási idő alapján csökkenő

sorrendbe rendez. A függvény végül visszaküldi a legfrissebb 20 értesítést, valamint az összes értesítés számát.

A getNotificationCount function a felhasználó értesítéseinek számát adja vissza három különböző kategóriából összesítve. Először lekéri, hogy hányan jelölték kedvencnek a felhasználó hirdetéseit. Ezután megszámolja a felhasználó eladóként kapott, még függőben lévő rendeléseket, majd ugyanígy a függőben lévő ajánlatokat is. A három értéket összeadja, és a kapott összesített számot JSON válaszban küldi vissza.

```
export const getNotificationCount = async (req, res) => {
  try {
    const userId = req.user.user_id;

    const favoriteCount = await prisma.favorites.count({
      where: {
        items: { user_id: userId }
      }
    });

    const orderCount = await prisma.orders.count({
      where: { seller_id: userId, status: 'pending' }
    });

    const offerCount = await prisma.offers.count({
      where: { seller_id: userId, status: 'pending' }
    });

    res.json({ count: favoriteCount + orderCount + offerCount });
  } catch (error) {
    console.error(error);
    res.status(500).json({ message: 'Szerver hiba' });
  }
};
```

```
export const createOffer = async (req, res) => {
  try {
    const io = getIO();
    const connectedUsers = getConnectedUsers();
    const item_id = parseInt(req.body.item_id);
    const { offer_price } = req.body;

    const existingOffer = await prisma.offers.findFirst({
      where: { item_id, buyer_id: req.user.user_id, status: 'pending' }
    });

    if (existingOffer) {
      return res.status(400).json({ message: 'Már van érvényes ajánlatod erre a termékre!' });
    }

    const item = await prisma.items.findUnique({
      where: { item_id },
      select: { user_id: true, price: true, title: true }
    });

    if (!item) {
      return res.status(404).json({ message: 'Hirdetés nem található' });
    }

    if (item.user_id === req.user.user_id) {
      return res.status(400).json({ message: 'Saját hirdetésre nem küldhetsz ajánlatot' });
    }

    const offer = await prisma.offers.create({
      data: {
        item_id,
        buyer_id: req.user.user_id,
        seller_id: item.user_id,
        offer_price: parseFloat(offer_price)
      }
    });

    const message = await prisma.messages.create({
      data: {
        sender_id: req.user.user_id,
        receiver_id: item.user_id,
        content: '',
        offer_id: offer.offer_id
      }
    });

    const sender = await prisma.users.findUnique({
      where: { user_id: req.user.user_id },
      select: { username: true }
    });

    const payload = {
      message_id: message.message_id,
      sender_id: req.user.user_id,
      receiver_id: item.user_id,
      content: '',
      offer_id: offer.offer_id,
      offer_status: 'pending',
      offer_price: offer_price,
      item_id: item_id,
      item_title: item.title,
      item_price: item.price,
      sender_name: sender.username,
      sent_at: new Date()
    };

    const receiverSocketId = connectedUsers.get(item.user_id);
    if (receiverSocketId) {
      io.to(receiverSocketId).emit('receive_message', payload);
    }

    const senderSocketId = connectedUsers.get(req.user.user_id);
    if (senderSocketId) {
      io.to(senderSocketId).emit('receive_message', payload);
    }

    res.json({
      message: 'Árajánlat elküldve!',
      offer_id: offer.offer_id
    });
  } catch (error) {
    console.error(error);
    res.status(500).json({ message: 'Szerver hiba' });
  }
};
```

A createOffer function egy új ajánlat létrehozását és annak azonnali elküldését végzi. Először kiolvassa a termék azonosítóját és az ajánlott árat, majd ellenőrzi, hogy a felhasználónak már van-e függőben lévő ajánlata ugyanarra a termékre; ha igen, hibát küld vissza. Ezután lekéri a hirdetés adatait, és ha a termék nem létezik, vagy a felhasználó a saját hirdetésére próbál ajánlatot tenni, szintén hibát jelez. Ha minden feltétel teljesül, a Prisma segítségével létrehoz egy új ajánlatot, majd egy ehhez kapcsolódó üzenetet is, amely üres tartalommal, de az ajánlat azonosítójával kerül mentésre. A függvény ezután lekéri a feladó felhasználónevét, összeállít egy részletes payload objektumot az ajánlat és az üzenet adataival, majd Socket.IO segítségével valós időben elküldi azt mind a címzettnek, mind a feladónak, amennyiben online vannak.

A acceptOffer function egy ajánlat elfogadását kezeli. Először kiolvassa az ajánlat azonosítóját az URL-ből, majd lekéri az adatbázisból azt az ajánlatot, amely a megadott ID-hez tartozik és amelynek az eladója a bejelentkezett felhasználó. Ha ilyen ajánlat nem létezik, vagy a felhasználónak nincs jogosultsága hozzá, hibát küld vissza. Érvényes ajánlat esetén a Prisma segítségével frissíti annak státuszát accepted értékre. Ezután, ha a vevő online van, a függvény Socket.IO segítségével valós időben értesítést küld neki az ajánlat státuszának megváltozásáról.

```
export const acceptOffer = async (req, res) => {
  try {
    const io = getIO();
    const connectedUsers = getConnectedUsers();
    const offerId = parseInt(req.params.offerId);

    const offer = await prisma.offers.findFirst({
      where: { offer_id: offerId, seller_id: req.user.user_id }
    });

    if (!offer) {
      return res.status(404).json({ message: 'Ajánlat nem található vagy nincs jogosultságod' });
    }

    await prisma.offers.update({
      where: { offer_id: offerId },
      data: { status: 'accepted' }
    });

    const receiverSocketId = connectedUsers.get(offer.buyer_id);
    if (receiverSocketId) {
      io.to(receiverSocketId).emit('offer_status_changed', {
        offer_id: offerId,
        status: 'accepted'
      });
    }

    res.json({ message: 'Árajánlat elfogadva!' });
  } catch (error) {
    console.error(error);
    res.status(500).json({ message: 'Szerver hiba' });
  }
};
```

A rejectOffer function egy ajánlat elutasítását végzi. Először kiolvassa az ajánlat azonosítóját az URL-ből, majd lekéri az adatbázisból azt az ajánlatot, amely a megadott ID-hez tartozik és amelynek az eladója a

```
export const rejectOffer = async (req, res) => {
  try {
    const io = getIO();
    const connectedUsers = getConnectedUsers();
    const offerId = parseInt(req.params.offerId);

    const offer = await prisma.offers.findFirst({
      where: { offer_id: offerId, seller_id: req.user.user_id }
    });

    if (!offer) {
      return res.status(404).json({ message: 'Ajánlat nem található vagy nincs jogosultságod' });
    }

    await prisma.offers.update({
      where: { offer_id: offerId },
      data: { status: 'rejected' }
    });

    const receiverSocketId = connectedUsers.get(offer.buyer_id);
    if (receiverSocketId) {
      io.to(receiverSocketId).emit('offer_status_changed', {
        offer_id: offerId,
        status: 'rejected'
      });
    }

    res.json({ message: 'Árajánlat elutasítva' });
  } catch (error) {
    console.error(error);
    res.status(500).json({ message: 'Szerver hiba' });
  }
};
```

bejelentkezett felhasználó. Ha ilyen ajánlat nem található, vagy a felhasználónak nincs jogosultsága hozzá, hibát küld vissza. Érvényes ajánlat esetén a Prisma segítségével frissíti annak státuszát rejected értékre. Ezután, ha a vevő online van, a függvény Socket.IO segítségével valós időben értesítést küld neki az ajánlat elutasításáról.

A counterOffer function egy visszaajánlat küldését kezeli. A kérésből kiolvassa a visszaajánlott árat és az ajánlat azonosítóját, majd lekéri az adatbázisból azt az ajánlatot, amely a megadott ID-hez tartozik és amelynek az eladója a bejelentkezett felhasználó. Ha ilyen ajánlat nem található, hibát ad vissza. A lekért ajánlathoz tartozó termék árának 70%-a alapján kiszámítja a megengedett legalacsonyabb visszaajánlati árat, és ha a felhasználó ennél kevesebbet adna meg, hibát küld a minimális értékkel együtt. Érvényes visszaajánlat esetén frissíti az ajánlat státuszát counter_offered értékre, és eltárolja a visszaajánlott árat. Ha a vevő online van, Socket.IO segítségével értesítést kap az ajánlat módosult állapotáról és a visszaajánlott összegről.

```
export const counterOffer = async (req, res) => {
  try {
    const io = getIO();
    const connectedUsers = getConnectedUsers();
    const { counter_price } = req.body;
    const offerId = parseInt(req.params.offerId);

    const offer = await prisma.offers.findFirst({
      where: { offer_id: offerId, seller_id: req.user.user_id },
      include: {
        items: { select: { price: true } }
      }
    });

    if (!offer) {
      return res.status(404).json({ message: 'Ajánlat nem található vagy nincs jogosultságod' });
    }

    const minPrice = Number(offer.items.price) * 0.7;
    if (parseFloat(counter_price) < minPrice) {
      return res.status(400).json({
        message: `A visszaajánlatod túl alacsony! Minimum ${Math.ceil(minPrice)} Ft lehet (max 30% kedvezmény).`,
        min_price: Math.ceil(minPrice)
      });
    }

    await prisma.offers.update({
      where: { offer_id: offerId },
      data: { status: 'counter_offered', counter_price: parseFloat(counter_price) }
    });

    const receiverSocketId = connectedUsers.get(offer.buyer_id);
    if (receiverSocketId) {
      io.to(receiverSocketId).emit('offer_status_changed', {
        offer_id: offerId,
        status: 'counter_offered',
        counter_price: counter_price
      });
    }

    res.json({ message: 'Visszaájánlat elküldve!' });
  } catch (error) {
    console.error(error);
    res.status(500).json({ message: 'Szerver hiba' });
  }
};
```

A deleteOffer function egy ajánlat törlését végzi. A kérés URL-paraméteréből kiolvassa az ajánlat azonosítóját, majd lekéri az adatbázisból azt az ajánlatot, amely ehhez az ID-hez tartozik és amelynek a vevője a

```
export const deleteOffer = async (req, res) => {
  try {
    const offerId = parseInt(req.params.offerId);

    const offer = await prisma.offers.findFirst({
      where: { offer_id: offerId, buyer_id: req.user.user_id }
    });

    if (!offer) {
      return res.status(404).json({ message: 'Ajánlat nem található vagy nincs jogosultságod' });
    }

    await prisma.offers.delete({ where: { offer_id: offerId } });

    const today = new Date().toISOString().split('T')[0];
    await prisma.users.updateMany({
      where: {
        user_id: req.user.user_id,
        last_offer_reset: new Date(today)
      },
      data: {
        daily_offers_count: { decrement: 1 }
      }
    });

    res.json({ message: 'Ajánlat törölve!' });
  } catch (error) {
    console.error(error);
    res.status(500).json({ message: 'Szerver hiba' });
  }
};
```

bejelentkezett felhasználó. Ha ilyen ajánlat nem található, hibát ad vissza. Érvényes ajánlat esetén törli azt az adatbázisból. Ezután meghatározza az aktuális nap dátumát, és ha a felhasználó napi ajánlatszámlálója még ugyanarra a napra vonatkozik, akkor egyet csökkent rajta.

A createOrder function egy termék megvásárlását kezeli. A kérésből kiolvassa a termék azonosítóját és a bejelentkezett felhasználó azonosítóját, majd lekéri az

```
export const createOrder = async (req, res) => {
  try {
    const item_id = parseInt(req.body.item_id);
    const buyer_id = req.user.user_id;

    const item = await prisma.items.findUnique({
      where: { item_id }
    });

    if (!item) {
      return res.status(404).json({ message: 'Termék nem található' });
    }

    if (item.status !== 'available') {
      return res.status(400).json({ message: 'Ez a termék már nem elérhető' });
    }

    if (item.user_id === buyer_id) {
      return res.status(400).json({ message: 'Nem vásárolhatod meg a saját termékedet' });
    }

    const order = await prisma.$transaction(async (tx) => {
      const newOrder = await tx.orders.create({
        data: {
          buyer_id,
          seller_id: item.user_id,
          item_id,
          status: 'pending'
        }
      });

      await tx.items.update({
        where: { item_id },
        data: { status: 'sold' }
      });

      return newOrder;
    });

    res.json({
      message: 'Vásárlás sikeres!',
      order_id: order.order_id
    });
  } catch (error) {
    console.error(error);
    res.status(500).json({ message: 'Szerver hiba' });
  }
};
```

adatbázisból a terméket. Ha a termék nem létezik, hibát ad vissza. Ha a termék státusza nem available, vagy ha a felhasználó a saját termékét próbálná megvenni, szintén hibát küld. Érvényes feltételek esetén tranzakciót indít, amelyben létrehoz egy új rendelést pending státusszal, majd frissíti a termék státuszát sold értékre.

A getMyOrders function a bejelentkezett felhasználóhoz tartozó rendelések listáját gyűjti össze. A

```
export const getMyOrders = async (req, res) => {
  try {
    const userId = req.user.user_id;

    const orders = await prisma.orders.findMany({
      where: { buyer_id: userId },
      include: {
        items: {
          include: {
            itemimages: {
              orderBy: [{ is_primary: 'desc' }, { display_order: 'asc' }],
              take: 1,
              select: { image_url: true }
            }
          }
        },
        users_orders_seller_idTousers: {
          select: { username: true, user_id: true }
        }
      },
      orderBy: { order_date: 'desc' }
    });

    const result = orders.map(o => ({
      order_id: o.order_id,
      item_id: o.item_id,
      order_date: o.order_date,
      status: o.status,
      title: o.items.title,
      price: o.items.price,
      description: o.items.description,
      seller_name: o.users_orders_seller_idTousers.username,
      seller_id: o.users_orders_seller_idTousers.user_id,
      image_url: o.items.itemimages[0]?.image_url || null
    }));

    res.json({ orders: result });
  } catch (error) {
    console.error(error);
    res.status(500).json({ message: 'Szerver hiba' });
  }
};
```

lekérdezés a felhasználó mint vevő összes rendelését visszaadja, és minden rendeléshez csatolja a kapcsolódó termék adatait, valamint a termék elsődleges képét. Emellett tartalmazza az eladó felhasználónevét és azonosítóját is. Az eredmények a rendelés dátuma alapján csökkenő sorrendben érkeznek. A kapott rekordokból a függvény egy egyszerűsített, egységes szerkezetű objektumlistát készít, amely tartalmazza a rendelés azonosítóját, a termék alapadatait, az eladó adatait és a termék képét.

```
export const getMySales = async (req, res) => {
  try {
    const userId = req.user.user_id;

    const sales = await prisma.orders.findMany({
      where: { seller_id: userId },
      include: {
        items: {
          include: {
            itemimages: {
              orderBy: [{ is_primary: 'desc' }, { display_order: 'asc' }],
              take: 1,
              select: { image_url: true }
            }
          }
        },
        users_orders_buyer_idTousers: {
          select: { username: true, user_id: true }
        }
      },
      orderBy: { order_date: 'desc' }
    });

    const result = sales.map(o => ({
      order_id: o.order_id,
      item_id: o.item_id,
      order_date: o.order_date,
      status: o.status,
      title: o.items.title,
      price: o.items.price,
      description: o.items.description,
      buyer_name: o.users_orders_buyer_idTousers.username,
      buyer_id: o.users_orders_buyer_idTousers.user_id,
      image_url: o.items.itemimages[0]?.image_url || null
    }));

    res.json({ sales: result });
  } catch (error) {
    console.error(error);
    res.status(500).json({ message: 'Szerver hiba' });
  }
};
```

A getMySales function a bejelentkezett felhasználó eladóként rögzített rendeléseit gyűjti össze. A lekérdezés minden olyan rendelést visszaad, ahol a felhasználó a seller_id, és ezekhez csatolja a kapcsolódó termék adatait, valamint a termék elsődleges képét. Emellett tartalmazza a vevő felhasználónevét és azonosítóját is. Az eredmények a rendelés dátuma alapján csökkenő sorrendben érkeznek. A kapott rekordokból a

függvény egy egységes szerkezetű objektumlistát készít, amely tartalmazza a rendelés azonosítóját, a termék alapadatait, a vevő adatait és a termék képét.

A updateOrderStatus function egy rendelés státuszának módosítását végzi. A kérés URL-paraméteréből kiolvassa a rendelés azonosítóját, a törzsből pedig az új státuszt. Ezután lekéri az adatbázisból a rendelést, és ellenőrzi, hogy a bejelentkezett felhasználó valóban az adott rendelés eladója-e. Ha a rendelés nem létezik, vagy a felhasználónak nincs jogosultsága a módosításhoz, hibát ad vissza. Jogosultság esetén frissíti a rendelés státuszát az adatbázisban.

```
export const updateOrderStatus = async (req, res) => {
  try {
    const id = parseInt(req.params.id);
    const { status } = req.body;
    const userId = req.user.user_id;

    const order = await prisma.orders.findUnique({
      where: { order_id: id },
      select: { seller_id: true }
    });

    if (!order) {
      return res.status(404).json({ message: 'Rendelés nem található' });
    }

    if (order.seller_id !== userId) {
      return res.status(403).json({ message: 'Nincs jogosultságod módosítani ezt a rendelést' });
    }

    await prisma.orders.update({
      where: { order_id: id },
      data: { status }
    });

    res.json({ message: 'Rendelés státusz frissítve' });
  } catch (error) {
    console.error(error);
    res.status(500).json({ message: 'Szerver hiba' });
  }
};
```

5. Felhasználói dokumentáció

1.1. A projekt háttere és a piaci rés. Az online adásvételi platformok és piacterek világa látszólag telített. A hazai felhasználók mindennapjaiba is beépültek az olyan nemzetközi óriások, mint a Vinted vagy az eBay, illetve a hazai viszonylatban évtizedes múltra visszatekintő Jófogás. Ha azonban mélyrehatóbban megvizsgáljuk a magyar e-kereskedelmi és apróhirdetési piacot, jól látható egy hatalmas, betöltetlen űr. A nemzetközi platformok ugyan technológiailag rendkívül fejlettek, működésük kiforrott, de sokszor hiányzik belőlük a kifejezetten hazai piacra szabott, lokális fókusz és a helyi közösségépítő erő. Ezzel szemben az egyetlen igazán nagy, magyar alternatíva, a Jófogás felett mára technológiai, vizuális és felhasználói élmény (UX) szempontjából is eljárt az idő. A felülete, az applikációs élménye és a tranzakciós funkciói elavultak, így már nem képesek maradéktalanul kiszolgálni a modern kor gyors, letisztult és mindenekelőtt biztonságos online vásárlási igényeit.

1.2. A Bellora víziója. A Bellora pontosan ezt a piaci hiányt felismerve és megragadva jött létre. Célunk, hogy egy olyan hazai fejlesztésű, innovatív alternatívát nyújtsunk a felhasználóknak, amely kompromisszumok nélkül egyesíti a legnépszerűbb nemzetközi platformok modern, felhasználóbarát funkcióit a hazai közösség egyedi igényeivel. A Bellora nem csupán egy újabb apróhirdetési oldal. Ez egy friss, átlátható és biztonságos magyar online tér, amely a 21. századi elvárásoknak megfelelően alkotja újra az online adásvétel élményét Magyarországon, fókuszba helyezve a bizalmat, a gyorsaságot és a mobil-első (mobile-first) megközelítést.

2. Fejezet: Technológiai Architektúra. A projekt fejlesztése során olyan modern technológiai alapokat kerestünk, amelyek mind a végfelhasználók, mind a fejlesztői csapat számára a legmagasabb minőséget, skálázhatóságot és kényelmet biztosítják.

2.1. Frontend technológia: React. A felhasználói felület (kliensoldal) megalkotásához a React könyvtárat választottuk. Dinamikus élmény: Segítségével gyorsan és hatékonyan építhetünk modern, interaktív Single Page Application (SPA) weboldalakat, ahol az oldalak közötti váltás azonnali, újratöltés nélkül történik. Komponens-alapú architektúra: Fejlesztői szempontból a React egy rendkívül kényelmesen és logikusan használható eszköz. Újrafelhasználható komponensekre épül, amely lehetővé teszi, hogy a kód tiszta és karbantartható maradjon. Reszponzivitás: A React használatával garantálni tudjuk, hogy a végeredmény egy villámgyors, minden kijelzőmérethez (mobil, tablet, asztali gép) tökéletesen alkalmazkodó felület legyen.

2.2. Adatbázis-kezelés: Prisma ORM. A háttérrendszer és az adatbázis közötti zökkenőmentes és típusbiztos kommunikációhoz a Prismát alkalmazzuk. Gördülékenység: Ennek a modern technológiának a legnagyobb előnye a páratlanul letisztult használat. A Prisma alkalmazásával a hagyományosan bonyolult SQL adatbázis-műveletek és komplex lekérdezések könnyedén

```
export const deleteAccount = async (req, res) => {
  try {
    await prisma.users.delete({
      where: { user_id: req.user.user_id }
    });

    res.json({ message: 'Fiók törölve' });
  } catch (error) {
    console.error(error);
    res.status(500).json({ message: 'Szerver hiba' });
  }
};
```

kezelhetővé válnak az alkalmazáskódon belül. Biztonság és stabilitás: Nemcsak a fejlesztési időt rövidíti le jelentősen, de egy stabil, hibamentes és az adatbázis-sémákat szigorúan követő alapot ad a projektnek, csökkentve a futásidejű hibák esélyét.

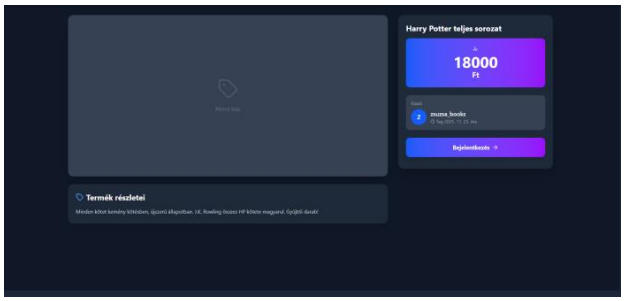
2.3. Backend technológia: Express és Node.js A szerveroldali logika a gyors és eseményvezérelt Node.js környezetben, az Express keretrendszerre épülve fut, amely iparági standardnak számít a skálázható REST API-k építésében.

3. Fejezet: A Rendszer Működésének Részletei

3.1. Kliensoldal (Frontend) Működése. A felhasználói böngészőben futó alkalmazás az alábbi technológiákra és elvekre épül: Komponensek: Moduláris React elemekből épül fel az oldal (pl. termékkártyák, navigációs sáv, űrlapok). Útvonalválasztás (Routing): A React Router felel az oldalak közötti zökkenőmentes navigációért, biztosítva a megfelelő URL struktúrát bejelentkezett és vendég felhasználók számára egyaránt. Adatkommunikáció: Az Axios HTTP kliens végzi a backend felé történő aszinkron API hívásokat (adatok lekérése, küldése).Állapotkezelés: A lokális és globális állapotok kezelése a beépített React Hook-ok (useState, useEffect) segítségével történik. Minőségbiztosítás: A Playwright end-to-end (E2E) tesztkörnyezet garantálja, hogy a legfontosabb felhasználói folyamatok (pl. vásárlás, regisztráció) minden frissítés után hibátlanul működjenek.

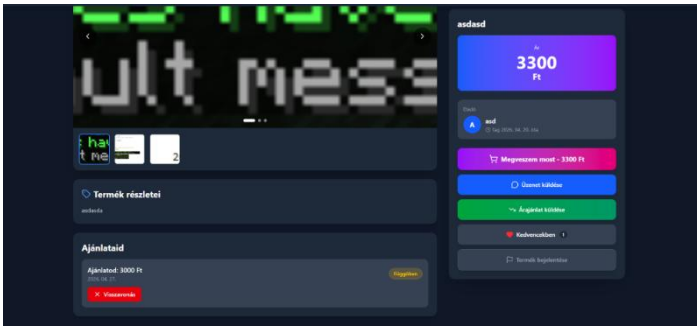
3.2. Szerveroldal (Backend) Működése. A központi szerver felelős az üzleti logikáért és az adatok védelméért: REST API: Szabványos végpontok (GET, POST, PUT, DELETE) az Express keretrendszerrel implementálva. Adatbázis réteg: A Prisma ORM végzi a közvetlen adatbázis-műveleteket. Köztesrétegek (Middleware-ek): A kérések feldolgozása előtt ezek az elemek végzik el az authentikációt, ellenőrzik a jogosultságokat és validálják a bejövő adatokat (hogy megakadályozzák a hibás vagy kártékony kéréseket). Kriptográfia: A felhasználói jelszavakat szigorúan bcrypt algoritmassal, "sózva" (salted) és hash-elve tároljuk az adatbázisban, így azok visszafejthetetlenek. Munkamenet kezelés: A bejelentkezés modern, állapotmentes JWT (JSON Web Token) alapú, amely biztonságos és gyors kommunikációt tesz lehetővé a kliens és a szerver között.

4. Fejezet: Felhasználói Szerepkörök és Funkciók. A rendszer funkciói a felhasználó aktuális státuszától függően válnak elérhetővé. Ezzel biztosítjuk, hogy a nyílt adatok mindenki számára láthatók legyenek a SEO (keresőoptimalizálás) és a vonzerő miatt, de a tranzakciók biztonságos, zárt környezetben történjenek.



4.1. Regisztráció nélkül (Vendégként) elérhető funkciók. A látogatók a platform megismerése érdekében számos funkciót használhatnak: Keresés és böngészés: A termékkatalógus teljes körű böngészése, szűrése kategóriák, árak és állapot szerint. Termék adatlapok: A hirdetések részletes adatainak (képek, leírás, ár, lokáció)

megtekintése. Profilok megtekintése: Az eladók nyilvános profiljának, korábbi értékeléseinek és aktív hirdetéseinek listázása a bizalomépítés érdekében.

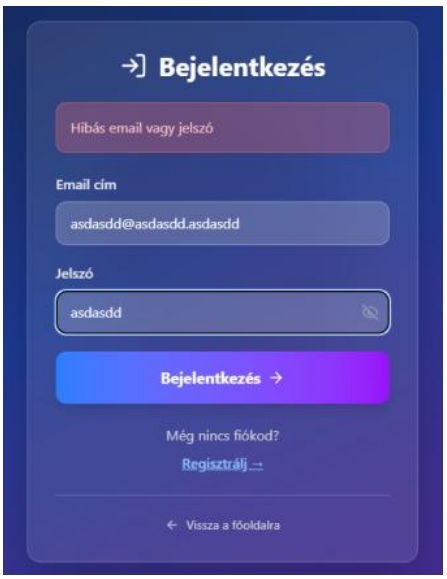


4.2. Regisztrált felhasználók számára elérhető funkciók. A bejelentkezést követően a felhasználók teljes jogú tagjává válnak a piactérnek: Hirdetéskezelés: Új hirdetés feladása képekkel és leírással, valamint saját hirdetések módosítása, törlése, szüneteltetése. Tranzakciós folyamatok: Ajánlat küldése (alku) egy eladónak, illetve a tényleges Vásárlás (rendelés/order létrehozása) folyamatának elindítása. Kommunikáció: Beépített, valós idejű üzenetküldő modul a vevők és eladók közötti egyeztetéshez. Személyre szabás: Kedvenc hirdetések mentése későbbi megtekintésre, saját felhasználói profil szerkesztése (profilkép, bemutatkozás, szállítási adatok).Közösségi funkciók: Sikeres vagy sikertelen tranzakció után értékelés (review) írása a partnerről, továbbá gyanús vagy szabálytalan hirdetések és felhasználók jelentése (report) az adminisztrátorok felé.

5. Fejezet: Adatbevitel és Validációs Szabályok. A rendszer stabilitásának és az adatbázis tisztaságának megőrzése érdekében minden felhasználói beviteli mezőt szigorú validációnak vetünk alá mind kliens, mind szerver oldalon.

Adatmező	**Formátum / Szabályok**	**Hosszúság**	**Karakterkészlet**
Felhasználónév	Nincs szóköz, egyedi érték	3 – 50 karakter	Csak betűk, számok, alulvonás. Ékezetes betűk nem ajánlottak.
Jelszó	Erős jelszó (Complex)	Min. 8 karakter	Kis- és nagybetű kötelező. Szám kötelező. Speciális karakter ajánlott.
E-mail cím	Szabványos e-mail formátum	N/A	@ és . karakterek megfelelő pozíciója (RFC 5322)
Termék címe	Beszédes, tömör megnevezés	Max. 150 karakter	Ékezetes betűk és alapvető írásjelek engedélyezettek.

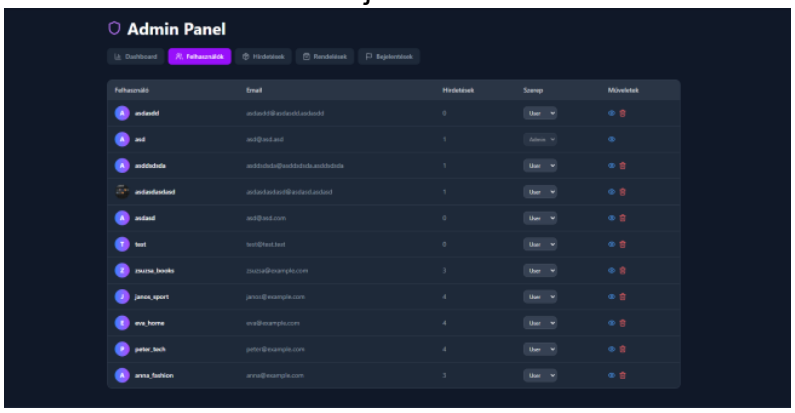
Adatmező	Formátum / Szabályok	Hosszúság	Karakterkészlet
Termék leírása	Részletes bemutatás	Max. 3000 karakter	Minden Unicode karakter, sortörések engedélyezettek.
Ár	Pozitív egész szám (HUF)	> 0 HUF	Csak számjegyek.
Üzenet szövege	Szabad szöveges chat	Max. 5000 karakter	Minden Unicode karakter engedélyezett (emoji támogatás).



6.1. Kliensoldali (Felhasználói) HibaüzenetekGyakori, képernyőn megjelenő értesítések (Toast notification vagy form hiba formájában):Autentikációs hiba: "Hibás felhasználónév vagy jelszó. Kérjük, próbálja újra!"Validációs hiba: "A mező kitöltése kötelező!" vagy "A jelszónak tartalmaznia kell legalább egy számot!"Jogosultsági hiba: "Nincs jogosultsága a kért művelet végrehajtásához."Erőforrás hiba: "A keresett termék nem található, vagy az eladó már törölte azt."Rendszerhiba: "A művelet végrehajtása sikertelen. Kérjük, ellenőrizze az internetkapcsolatot, vagy próbálja újra később."

6.2. Backend HTTP StátuszkódokA kliens és a szerver közötti kommunikáció szabványos HTTP kódokra épül a fejlesztői átláthatóság érdekében:200 OK / 201 Created: A kérés sikeresen lefutott, az adat létrejött.400 Bad Request: Hibás kérés (pl. rosszul formázott adatok, validációs hiba a szerveroldalon).401 Unauthorized: A felhasználó nincs bejelentkezve, érvénytelen vagy lejárt a JWT token.403 Forbidden: A felhasználó be van jelentkezve, de nincs joga a művelethez (pl. más felhasználó hirdetését próbálja módosítani).404 Not Found: A kért erőforrás (felhasználó, termék, végpont) nem létezik.500 Internal Server Error: Váratlan szerveroldali vagy adatbázis hiba.

7. Fejezet: Adminisztrátori Felület és Moderáció. A platform tisztaságának, biztonságának és a szabályzatok betartásának felügyeletére egy dedikált Adminisztrátori Vezérlőpult (Dashboard) készült. Ez a felület szeparáltan, egy külön (védett) útvonalon (route) érhető el, és kizárólag a legmagasabb, "admin" szerepkörrel rendelkező JWT tokennel rendelkező fiókok használhatják.



7.1. Adminisztrátori Jogosultságok. Az adminisztrátorok a normál felhasználói funkciókon felül az alábbi kiemelt jogosultságokkal rendelkeznek: Felhasználókezelés (User Management): A regisztrált felhasználók teljes listájának megtekintése, keresés a bázisban. Szabályszegő felhasználók figyelmeztetése, felfüggesztése (ban) vagy végleges törlése a platformról. Tartalommoderáció: Bármely hirdetés szerkesztése vagy azonnali törlése, amennyiben az sérti a platform szabályzatát (pl. tiltott termékek, illegális tartalom, spam).Jelentések (Reports) kezelése: A közösség által beküldött jelentések (report ticketek) feldolgozása. Az adminisztrátor ezeket megvizsgálhatja, majd dönthet a jóváhagyásról (szankció alkalmazása) vagy az elutasításról (vaklárma). Taxonómia és Kategóriák: A platform termékkategóriáinak és alkategóriáinak (pl. Ruházat, Elektronika -> Mobiltelefonok) dinamikus kezelése, új kategóriák létrehozása a piaci trendeknek megfelelően. Analitika és Statisztikák: Átfogó képet kapnak a platform működéséről. Láthatják a napi új regisztrációk számát, a feladott hirdetések volumenét, az aktív tranzakciókat és a hibaarányokat, ami segíti az üzleti és fejlesztési döntéshozatalt.

6. Összefoglalás

Összességében sikeresnek és jól strukturáltnak értékelem a projektet. A fejlesztés során törekedtem arra, hogy a rendszer minden fő funkciója stabilan, átláthatóan és felhasználóbarát módon működjön, miközben a háttérben egy jól szervezett, biztonságos és bővíthető architektúra szolgálja ki az alkalmazást. A backend és a frontend komponensek összehangoltan működnek, az adatbázis-struktúra pedig megfelelő alapot biztosít a piactér logikájához. A tesztek sikeres lefutása megerősítette, hogy a rendszer megbízhatóan kezeli a különböző felhasználói útvonalakat, hibákat és jogosultsági szinteket. Úgy érzem, hogy a projekt során megszerzett tapasztalatok és a végeredmény egyaránt tükrözik a befektetett munkát és a tudatos tervezést.

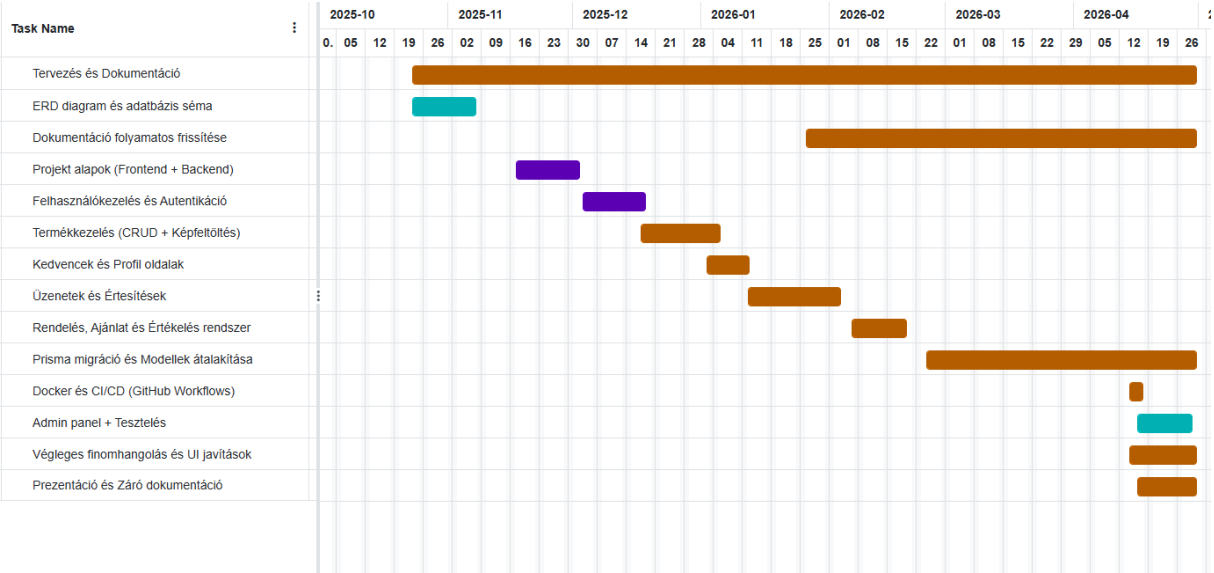
A program jövőbeli hasznosulását tekintve úgy látom, hogy a Bellora rendszer alapjai könnyen továbbfejleszthetők és valós környezetben is alkalmazhatók. A piactér logikája, a felhasználói interakciók, az ajánlatkezelés, a rendelési folyamat és az adminisztrációs felület mind olyan funkciók, amelyek egy éles szolgáltatás alapját is képezhetik. A projekt alkalmas arra, hogy később új funkciókkal bővítsem – például értesítési rendszerrel, mobilalkalmazással, fejlettebb keresővel vagy fizetési integrációval. A vizsgától függetlenül is értékes alapot jelent számomra, hiszen a teljes fejlesztési folyamatot végigjárva egy olyan rendszert hoztam létre, amely valós felhasználói igényeket is képes lenne kiszolgálni.

Munkamegosztás:

Narancs – Közös

Lila – Csák Zoltán Barnabás

Kék – Gere Dániel



7. Irodalom jegyzék

- Dokumentáció felépítésére, tartalmára minta: https://infojegyzet.hu/webszerkesztes/dokumentacio/?authuser=1
- Számos kódolási oktatóanyag: https://www.w3schools.com
- React-hoz használt útmutató: https://react.dev
- Vite-hez használt útmutató: https://vite.dev
- Tailwind-hez használt útmutató: https://tailwindcss.com
- Prisma-hoz használt útmutató: https://www.prisma.io
- Socket.IO-hoz használt útmutató: https://socket.io
- Jest-hez használt útmutató: https://jestjs.io
- Express-hez használt útmutató: https://expressjs.com
- Multer-hez használt útmutató: https://www.npmjs.com/package/multer